Deccan Kitchen & Co.

Technical design document

GitHub: https://github.com/DhanushYarragunta/direct-order-platform

Review candidate | 3 November 2026 | Company A, Hyderabad

Live app: not deployed. Run locally at http://127.0.0.1:3000 using deployment.md. This document describes the working local candidate; the latest changes must be committed and pushed by the owner before submission.

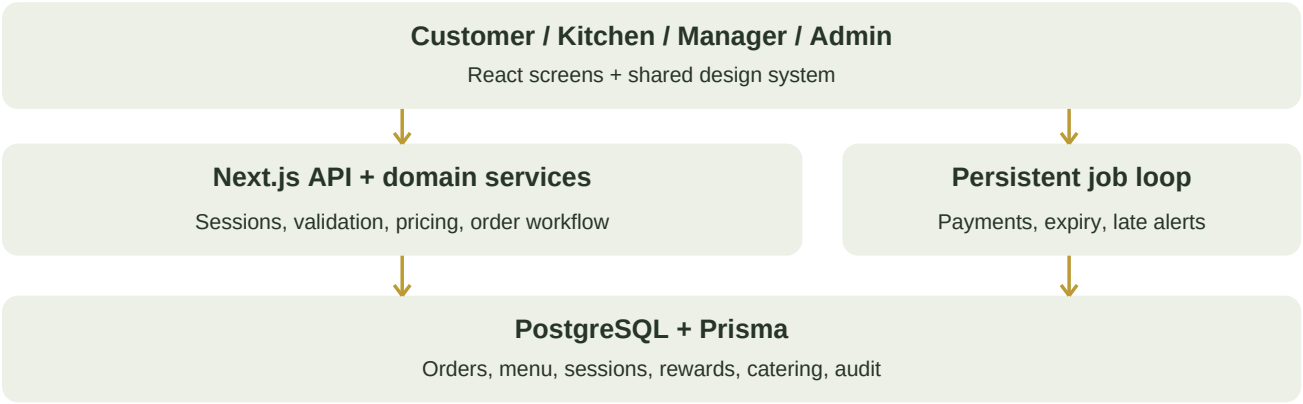
Product and scope

Company A wants to own its direct ordering channel, control its menu centrally and send orders to the right kitchen. The original CLIENT_BRIEF.md is the requirements authority. The client estimates an annual benefit of about INR 1.14 crore; this is a business-case assumption, not a measured result of this build.

The implemented core covers browsing, portions and add-ons, location-based outlet selection, a persistent cart, test UPI/card payment, live tracking, menu administration and outlet kitchen tickets. This candidate also includes coupons, loyalty, party requests, city price overrides, scheduled menus, reorder suggestions and outlet reporting.

Architecture and why

One Next.js 16 App Router application serves customers, kitchen staff, managers and central administrators. TypeScript spans UI and domain logic. PostgreSQL with Prisma 7 provides shared state and transactional guarantees. Separate HTTP handlers, domain modules and UI components keep changes reviewable without introducing several services.



React screens use ordinary authenticated JSON requests for mutations. Server-sent events announce changed data; clients fetch authoritative state and also poll every 12 seconds. The three-second job loop handles delayed payment settlement, expiry and late alerts from durable database timestamps. A long-running Node host is required.

Data and main flows

Data model

- User and Session: role, optional staff outlet, consent, scrypt password hash, hashed opaque session token and expiry. Registration always creates a customer.

- Outlet, MenuItem, MenuCityPrice, Availability and AddOn: shared catalogue, city overrides, India-time serving windows and separate per-outlet stock state.

- Order and OrderLine: immutable price/name/option snapshots, fulfilment details, totals, coupon and points snapshots, payment-confirmed time, preparation promise and version.

- Payment: unique attempts, simulated outcome and durable delayed-settlement deadline. Message and Audit record customer updates and operational actions.

- Coupon and LoyaltyEntry: configured offer rules and an append-only points ledger. Order plus ledger kind is unique, preventing duplicate debit, release or earnings.

- CateringRequest: owner, outlet, event, party requirements, manager response, version and unique client retry key. RateLimit stores bounded login attempts.

Customer checkout to kitchen

1. The selected outlet's effective menu includes city prices and serving-hour availability. The user explicitly chooses a dish, portion and extras.

2. The quote endpoint calculates discounts, points and GST from current database prices. No client-supplied price is accepted.

3. Checkout validates delivery radius and stock, locks the customer and outlet, and stores order/line snapshots plus any points debit in one serializable transaction.

4. A unique customer/checkout key and canonical payload hash allow safe retries. Reusing a key for a changed cart is rejected.

5. The test gateway simulates success, failure or an eight-second delayed result. A successful settlement rechecks availability, schedule and first-order coupon eligibility. A failed recheck refunds the simulated payment and restores points.

6. Paid orders become PLACED, then PREPARING, READY and HANDED_OFF. Version checks reject stale kitchen actions. Customers may cancel while PLACED; payment and points are reversed together.

Money and concurrency

All prices are integer paise. Displayed prices include 5% GST; the included tax is round(total x 5 / 105). Discounts apply before the included-tax calculation. Fifteen-minute unpaid expiry releases reserved rewards. Failed payment attempts preserve the checkout for retry until expiry. Database row locks, unique constraints and bounded serialization/deadlock retries protect overlapping requests and workers.

Automation and extended workflows

Preparation estimates and recovery

The estimator is a transparent rules-based baseline, not a trained AI model. Suggested minutes = ceil(10 + new order work / 2 + queued work / active stations). Work includes dish preparation, quantity, family portions and extras. The promise is the higher of the suggestion and the outlet minimum.

Invalid numeric inputs fall back to the configured minimum, or 30 minutes when that minimum is invalid. The estimate and fallback flag are stored. Managers can override an active promise, respecting the outlet minimum, with a required reason and audit entry. At ten minutes past the current promise, the job loop creates a deduplicated manager alert and simulated customer message. No external AI service can block checkout.

Coupons and direct-order rewards

Demo policy assumptions: WELCOME100 gives INR 100 off a first paid order with an item subtotal of at least INR 499. BIRYANI10 gives 10% off a subtotal of at least INR 399, capped at INR 150. One point equals INR 1; redemption is capped at 20% of the amount after the coupon. Handoff earns one point per full INR 100 actually paid.

The user row serializes balance changes across outlets. Pending welcome-code checkouts reserve eligibility; settlement checks again if another order was paid. A checkout debits reserved points; cancellation, expiry or stock refund releases them exactly once. Earnings occur exactly once at handoff. The wallet shows the balance and recent ledger entries.

Menu rules and suggestions

An outlet's city selects an optional price override; absent overrides use base prices, and absent family overrides inherit the base family price. Prices on existing orders stay unchanged. Schedules use Asia/Kolkata and selected start weekdays; overnight windows belong to their starting day. Public availability combines publication, kitchen open state, serving hours and stock. Staff toggle stock independently of time-based availability.

Suggestions rank the customer's purchased items from the latest 50 paid orders at the selected outlet, excluding unavailable dishes. They open customization only after an explicit click and never add anything automatically.

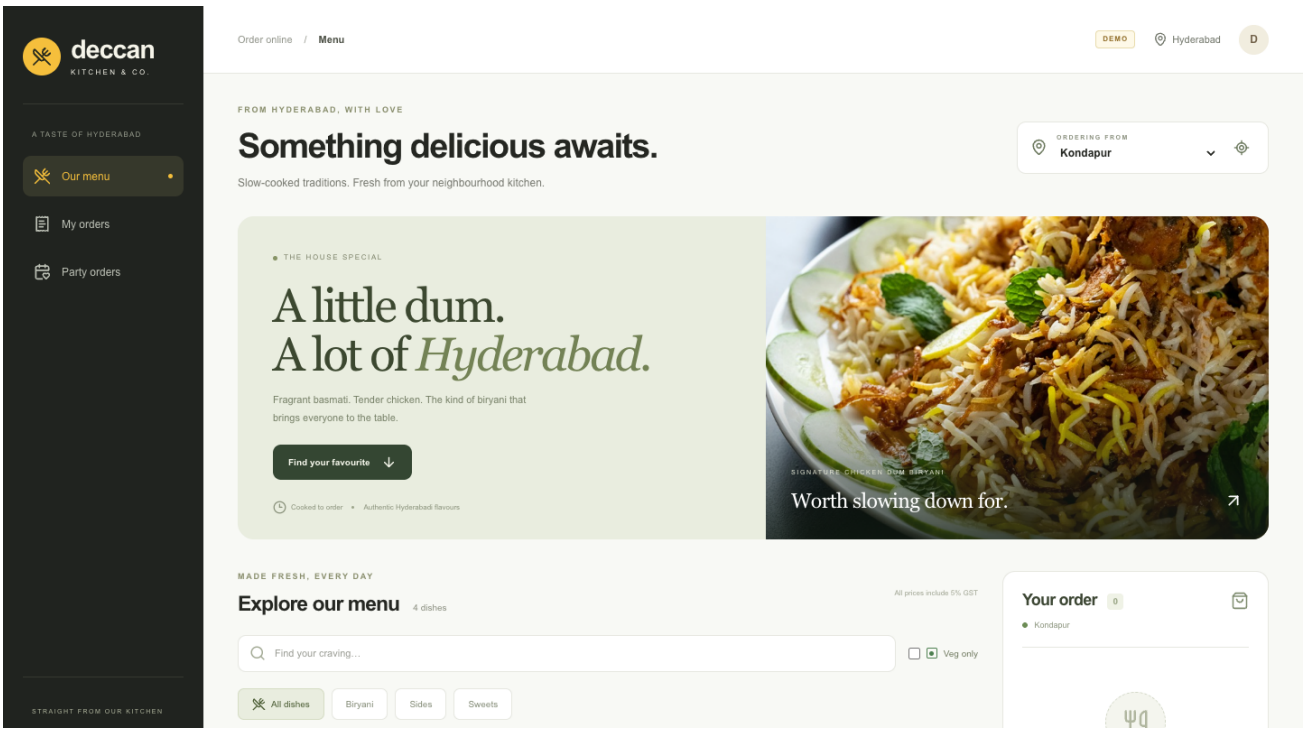
Party orders

Assumptions: 10-500 guests, at least 24 hours' notice, at most 180 days ahead. A request collects venue, menu, dietary requirements, contact and optional budget. A scoped manager or admin confirms or declines with a note; the customer sees the response and can cancel while awaiting it. Submission retries are idempotent. Catering creates no automatic quote, payment or kitchen ticket.

Customer experience and design

Food first, short path to checkout

The concept brand uses a charcoal navigation rail, warm amber actions, light surfaces and generous spacing. Four supplied food photographs provide the menu's visual focus. Source marks are retained; image rights are not implied. Category/search and vegetarian filters keep the menu easy to scan. Every dish exposes diet, spice and allergen details.



Working local application - browser verification capture.

The cart persists IDs/options/quantities rather than trusted prices. Switching outlets loads a fresh menu; stale responses cannot overwrite a newer outlet request. Checkout asks for fulfilment and a phone number, with an address and location check for delivery. Coupon and loyalty controls sit together, and the server quote appears in the same order summary.

Supporting screens

Payment explicitly identifies itself as a simulation. Tracking combines stages, current promise, a receipt and simulated messages. Order history includes rewards activity. Party requests have their own form and manager-response cards. Error messages include retry actions; empty states explain the next step. Sign-in exposes labelled demo roles only when demo mode is enabled.

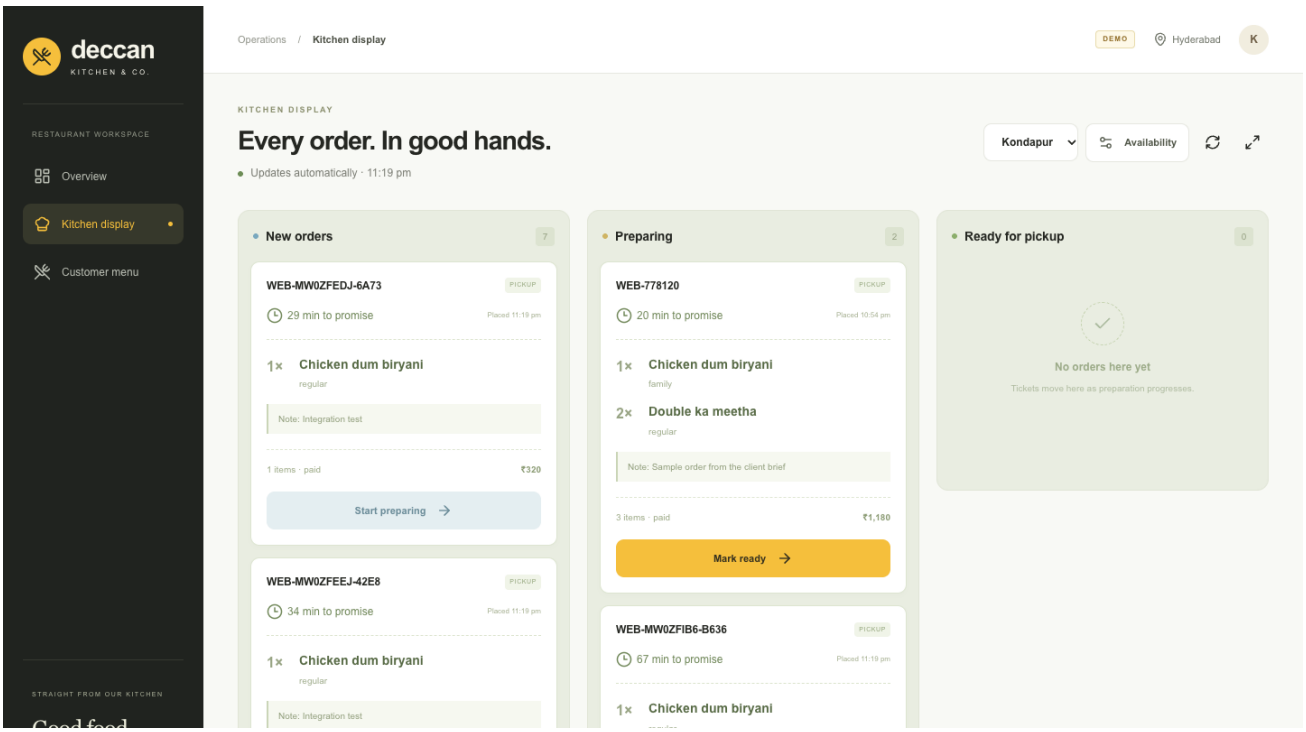
Responsive and accessible choices

Customer layouts collapse to one column on phones. Menus use named buttons, food text accompanies diet symbols, and statuses use words as well as colour. Radix dialogs manage focus; keyboard focus and reduced-motion styles are present. Kitchen actions are large and stable. Browser overflow checks and screenshots cover narrow screens; physical two-metre readability still needs a real kitchen-tablet evaluation.

Operations and reporting

Kitchen and central control

Each kitchen account is tied to one outlet. Its display separates received, preparing and ready tickets, highlights overdue work and supports handoff. Managers can revise promises with an explanation. Central administrators edit dishes, supplied-library photos, prices, city overrides, serving windows and outlet availability.



Working local application - browser verification capture.

Reports with explicit definitions

The overview queries all matching database rows, with no 100-order source cap. Admins select an outlet or all kitchens; staff are restricted to their assigned outlet. Rolling 24-hour, 7-day and 30-day periods cover orders, net paid sales, average preparation, refunded cancellations and repeat customers. Daily activity uses India Standard Time.

Repeat customers means at least two paid orders in the selected period. Outlet rows count visits to that outlet; the combined metric deduplicates customers across outlets. This is not a cohort-retention measure. Unpaid/expired checkouts are distinguished from refunded cancellations. Preparation runs from successful payment to READY; historical sample orders use their original placed time as the best available approximation.

Delivery and operating model

The system routes orders by location/delivery radius but does not dispatch real riders. Handoff is a kitchen stage. It does not replace billing or inventory, and has no delivery-app integration. Payments and WhatsApp/SMS messages are simulated, as permitted by the brief. Party confirmation is human-operated and remains separate from live kitchen tickets.

Verification and next steps

Reproducible verification

Verified on 3 November 2026: TypeScript, 36 domain tests, 27 PostgreSQL integration tests, nine browser scenarios and the production build passed. Run npm run verify:core to reproduce migration/seed and all checks. The temporary database is dropped afterwards; the working demo is preserved. Evidence is stored in .local/core-verification.json and .local/verification.log, with screenshots under .local/screenshots and failure traces under test-results.

Coverage targets pricing/GST, request validation, password hashing, radius routing, estimate fallback, promises, overnight schedules, coupon caps and catering notice. Database tests exercise concurrent retries, refunds, points balance safety, first-order eligibility, expiry, late alerts, customer/outlet isolation, stale versions and uncapped report totals. Browser checks cover desktop/mobile ordering, payment through kitchen handoff, menu changes, rewards reversal, party confirmation and reporting.

Security and personal data

Session tokens are random, cookie-held and hashed at rest; cookies are HttpOnly and SameSite=Lax, with Secure on HTTPS origins. Mutations enforce the exact application origin, bounded JSON bodies and strict Zod schemas. Authorization is repeated on the server. Passwords use salted scrypt. Marketing consent is optional and unticked by default; demo messages are transactional. Secrets, local databases, build output and reports are ignored by Git.

Assumptions to review

The brand, outlet coordinates/addresses, five-kilometre radius, menu prices, preparation work/capacity, reward and coupon policies, catering limits and additional historical records are invented implementation assumptions. The three original sample order IDs, totals and states are preserved. Three outlets and four photographed active dishes deliberately keep the demonstrable menu compact. All seeded outlets are in Hyderabad; city pricing is implemented but other-city expansion is not asserted.

FSSAI values are conspicuous demo placeholders, not licences. Sample accounts use fake .example email addresses. Email/password replaces the earlier proposal's mock OTP. The estimator is a baseline whose accuracy needs real operational data. Current sample data and demo shortcuts are unsuitable for production.

Limits and next steps

Public hosting, real payments/messages/delivery, verified licences and photo rights, retention/deletion policies, and peak-load measurement are not completed. The target 1,200 orders/hour and sub-60-second median checkout have not been benchmarked. There is no claim that client adoption or repeat-customer targets have been achieved. Image uploads remain a supplied-library picker; coupon policies are seeded rather than managed through a marketing CMS.

After design and architecture review: calibrate preparation estimates against measured work, validate accessibility on target tablets, run load tests, and select a long-running Node/PostgreSQL host. Then integrate approved providers and operational monitoring. The owner should commit and push this candidate, verify public repository access, and submit the two-file ZIP containing this PDF and deployment.md.